

Performance Analysis of GIS-Style Raster Processing

1. Project Overview

I used GIS-style raster processing as a scalable C++ workload. The project has two main operations:

- 3×3 smoothing: each interior cell becomes the average of its 3×3 neighbourhood.
- Threshold classification: each cell becomes 0 or 1 depending on whether it is above a threshold.

For smoothing, I compared: • scalar row-major traversal, • scalar column-major traversal, • manual 256-bit AVX SIMD.	For classification, I compared: • an explicit if/else branch, • a branchless comparison.
--	--

Raster values are stored in a contiguous `std::vector<float> values` using row-major indexing. [Code: `src/raster.h` -> `std::vector<float> values; src/raster.cpp` -> `smooth_row_major_into()`]

The benchmark raster is generated by my C++ code using:

`value = (index % 1000) / 1000.0`

So, the same dimensions always generate the same input. No external GIS dataset is needed. [Code: `src/raster.cpp` -> `generate_synthetic_raster()` and `static_cast<float>(index % 1000) / 1000.0f`]

For branch testing, I generated values of 0.25 and 0.75 around a threshold of 0.5, with 5%, 50%, or 95% of values passing, in either grouped or shuffled order. [Code: `src/raster.cpp` -> `generate_threshold_raster()`]

2. How I Evaluated It

All final performance experiments were performed on the same CSIL machine:

CPU: Intel Core i7-9700K @ 3.60 GHz OS: Ubuntu 24.04.4 LTS, x86-64 Compiler: g++ 13.3.0	Cache: 32 KiB L1 data/core, 256 KiB L2/core, 12 MiB shared L3 SIMD support: AVX, AVX2, FMA
---	---

The main benchmark setup was:

- input generation and output allocation happened before timing,
- kernel time used `clock_gettime(CLOCK_MONOTONIC)`,
- timing experiments used 9 runs and the median,
- final runs were pinned to CPU 0 with `taskset -c 0`,
- `perf stat` was used for cache and branch counters.

[Code: `src/main.cpp` -> `clock_gettime()`; benchmark kernels: `smooth_row_major_into()`, `smooth_column_major_into()`, `smooth_simd_avx_into()`]

I tested raster sizes from 64×64 to 2048×2048, so the largest data was well beyond the CPU's L1/L2 cache sizes.

3. Results

The same row-major smoothing code was compiled with different optimization settings.

Build	2048×2048 median
-O0	87.535 ms
-O2	8.216 ms
-O3	2.003 ms
-O3 -fno-tree-vectorize	8.369 ms

At 2048x2048, -O3 was about 43.7x faster than -O0 and 4.18x faster than -O3 -fno-tree-vectorize. The full 43.7x gain is not only from SIMD because -O3 also enables other optimizations.

Representative run: make clean make OPT=-O3 taskset -c 0 ./build/raster-performance row 2048 2048 9	GCC also directly reported: optimized: loop vectorized using 32 byte vectors
--	---

The optimized assembly contained packed AVX instructions such as vaddps and vdivps.

Manual SIMD

The manual version uses 256-bit AVX. Since each float is 32 bits, one vector can process 8 floats at once.

[Code: src/raster_simd.cpp -> smooth_simd_avx_into[]]

At 2048x2048:

Version	Median
Scalar, vectorization disabled	8.477 ms
Manual AVX	2.155 ms
GCC auto-vectorized	1.940 ms

This showed me that handwritten SIMD can yield significant speedups, but handwritten code is not automatically better than what the compiler produces.

Memory Locality

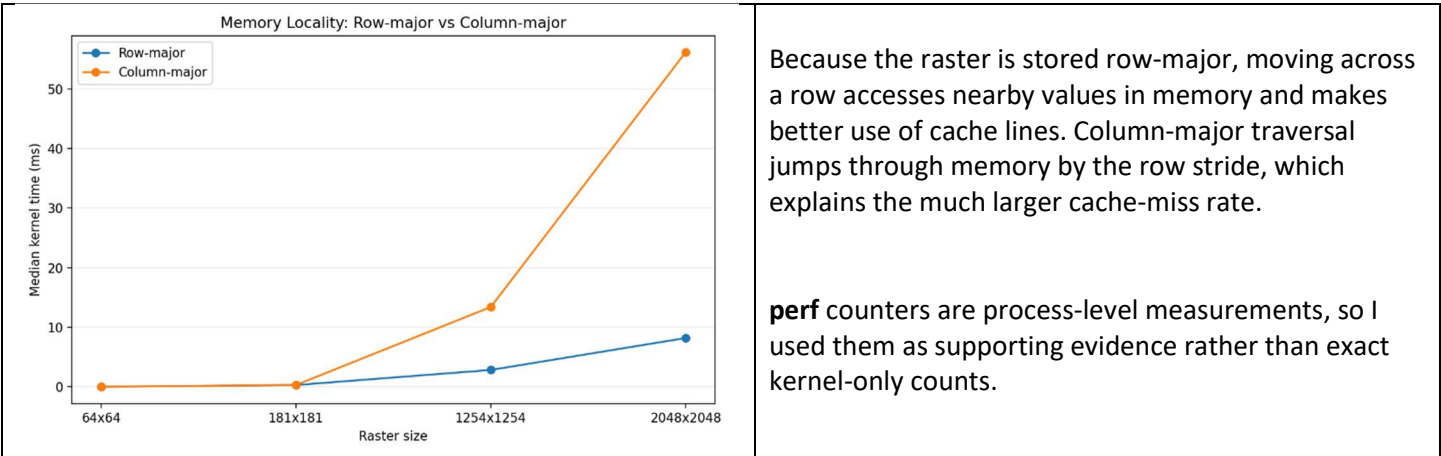
Row-major and column-major smoothing do the same arithmetic. Only traversal order changes. [Code: src/raster.cpp -> smooth_row_major_into(and smooth_column_major_into()]. The locality comparison used -O3 -fno-tree-vectorize to reduce compiler SIMD as a confounding factor.

At 2048x2048:

Traversal	Time	L1 miss rate
Row-major	8.187 ms	1.999%
Column-major	56.143 ms	38.475%

Representative runs:

taskset -c 0 ./build/raster-performance row 2048 2048 9	taskset -c 0 ./build/raster-performance column 2048 2048 9
---	--



Branch Prediction

The classification code was tested using grouped and shuffled inputs.

[Code: `src/raster.cpp` -> `classify_branch_into()` and `classify_branchless_into()`]

Assembly confirmed the intended difference:

explicit branch: <code>vcomiss + ja</code>	branchless: <code>vcomiss + seta</code>
--	---

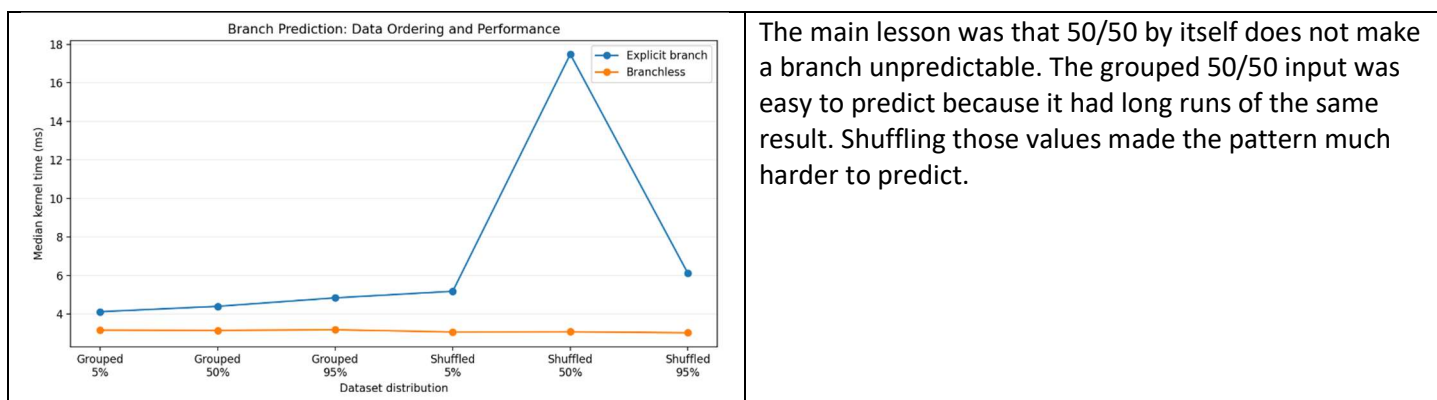
The strongest result was the shuffled 50% input:

Version	Time	Branch miss rate
Explicit branch	17.490 ms	20.356%
Branchless	3.075 ms	0.051%

The branchless version was about 5.69× faster. For comparison, the grouped 50% explicit branch took only 4.398 ms with a 0.021% miss rate.

Representative commands:

<code>taskset -c 0 ./build/branch-benchmark branch 2048 2048 50 shuffled 9</code>	<code>taskset -c 0 ./build/branch-benchmark branchless 2048 2048 50 shuffled 9</code>
---	---



4. Correctness and Memory Check

I ran correctness tests before relying on the performance results. make test passed the final test suite, including comparisons between the smoothing/classification implementations and SIMD remainder cases.

[Code: `tests/tests.cpp` -> `test_smooth_simd_width_cases()` and `test_classification_threshold_rule()`]

Run: <code>make test</code>	Valgrind was also run on the test executable: <code>valgrind --leak-check=full --show-leak-kinds=all --error-exitcode=1 ./build/tests</code>
--------------------------------	---

It reported: **0 bytes in use at exit** and **0 errors**

5. What I Learned

The main results were:

- Compiler optimization had a huge effect: -O3 was about 43.7× faster than -O0.
- Auto-vectorization mattered: disabling it made the largest smoothing test about 4.18× slower.
- Manual AVX gave about a 3.93× speedup, but GCC was still slightly faster on the largest raster.
- Memory order mattered a lot once the data became large: column-major was about 6.86× slower and had far more cache misses.
- Branches were slow when their outcomes were hard to predict. Shuffled 50/50 data made the branchless version about 5.69× faster.

Overall, this project showed me that performance is not just about the C++ algorithm itself. Compiler choices, SIMD instructions, memory layout, caches, and branch predictability can all change the speed of the same basic computation.